DATA DIVERGENCE | TRACK 1

Machine Failure Prediction

Methodology Summary - Industrial Intelligence & Smart Manufacturing

ZBH LOOP
Gokul | Aakash

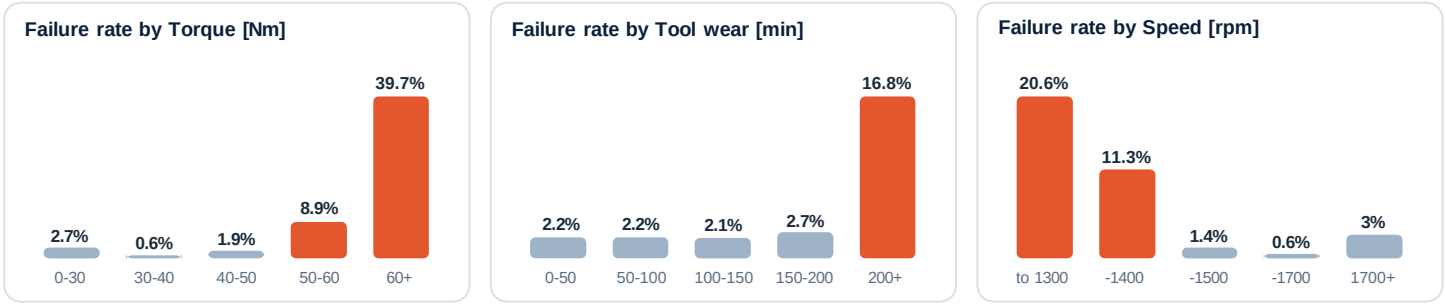
0.904	0.881	99.1%	79.3%
PR-AUC	**F1-score**	**Precision**	**Recall**
main ranking metric	failure (minority) class	2 false alarms in 7,999 machines	215 of 271 failures caught

In one line: when our model raises an alarm it is right 99% of the time, and F1 rose from a 0.74 baseline to 0.88 through engineered features and a blended model.

1. Data snapshot

7,999	6	271	0	L 60% | M 30% | H 10%
machine records	input signals	failures (3.39%)	missing values in train.csv	product quality Type mix

2. Exploratory data analysis - what we found



- **Severe class imbalance.** Only 3.39% of records are failures, so plain accuracy is misleading (always predicting "healthy" scores 96.6%). We optimise and report PR-AUC and F1 on the failure class.
- **Risk explodes at the extremes.** Torque above 60 Nm: 73 of 184 machines failed (40%). Tool wear above 200 min: 101 of 602 failed (17%) versus about 2% below. Speed at or under 1300 rpm: 35 of 170 failed (21%).
- **A hidden power band.** Mechanical power (torque x speed) below 3.5 kW or above 9 kW: all 73 of 73 machines failed. This motivated our torque x speed feature.
- **Failing machines look different on average:** torque 49.6 vs 39.6 Nm, tool wear 145 vs 107 min, a smaller process-air temperature gap (9.4 vs 10.0 K). Type L fails most (3.8%), Type H least (2.2%).

3. Feature engineering - 3 physics-based features

Temperature gap
Process temp - Air temp (K)
Separates heat generated by the process from ambient conditions.

Torque x Speed
Mechanical power proxy
Failures cluster at very low and very high power; 73 of 73 machines outside the band failed.

Torque x Tool wear
Load on an ageing tool
Combines two of the top-4 drivers to capture overstrain of worn tools.

4. Model & pipeline - everything reproducible in one script

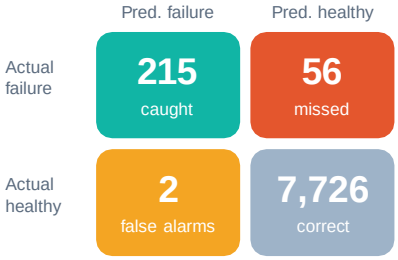
Raw input	Fill blanks	Prepare	Blend	Threshold	Output
6 columns exactly as in train.csv	median for numbers, most frequent for Type	scale numbers, one-hot Type, add 3 engineered features	Gradient Boosting + Random Forest (soft vote)	failure if probability >= 0.6526	probability + Healthy / Potential failure

All imputing, scaling and feature creation sit **inside the saved model**, so any CSV with the same six columns (blanks or unseen Type values included) is handled automatically. Libraries and versions used: Python 3.12, scikit-learn 1.9.1, pandas 3.0, numpy 2.4 (no other packages).

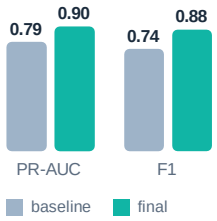
5. Validation - honest scores on unseen rows

- **Stratified 5-fold cross-validation** (shuffled, seed 42) keeps the 3.4% failure ratio in every fold; each machine is scored by a model that never saw it.
- **Metrics = the organisers' metrics:** PR-AUC and F1 on the failure class. Accuracy (99.3%) is shown only for context.
- **Threshold** 0.5016 was tuned to maximise F1 on the out-of-fold predictions, so scores may be slightly optimistic; the hidden test set can differ.
- **Final model** is refit on all 7,999 rows. Baseline = one boosted-tree model on the 6 raw columns with the same protocol.

Cross-validated results (7,999 machines)

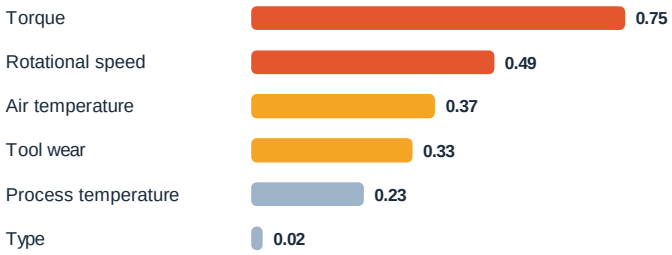


Baseline vs final



6. Interpretability - what drives a failure alarm

Permutation importance (drop in PR-AUC when shuffled)



- **Torque and speed dominate.** Together they define the power band where failures concentrate.
- **Heat matters through the gap.** Air and process temperature rank 3rd and 5th, partly because the model uses their difference.
- **Tool wear acts as a threshold:** risk jumps past about 200 minutes instead of rising gradually.
- **Product Type barely matters,** so alarms are driven by physical signals, not labels. Measured on a held-out 25% split.

7. Practical business takeaways

Watch the extremes

Flag power outside 3.5-9 kW (all 73 such machines failed), torque above 60 Nm, or speed at or below 1300 rpm.

Smart tool changes

Failure rate is about 17% past 200 min of wear versus about 2% before. Replace tools by condition, not an arbitrary schedule.

Alarms to trust

About 99% of alerts are real (2 false alarms per 7,999 machines), so crews will act on them and avoid alarm fatigue.

Know the blind spot

56 failures (21%) look healthy on these six signals. Add vibration or acoustic sensors, or lower the threshold if downtime costs more than inspections.

Limitations

- Scores are local cross-validation estimates; the hidden test set may differ.
- The threshold was tuned on the same out-of-fold predictions, so results can be slightly optimistic.
- train.csv has no timestamps, so a time-based split was not possible.

Next steps

- Set the threshold from real downtime vs inspection costs (a lower threshold catches more failures).
- Add vibration or acoustic sensing to reach the failures that look healthy today.
- Re-validate on the organisers' hidden test data and retrain as new telemetry arrives.

How to use the model and Structure

Install the requirements with pip install scikit-learn pandas numpy joblib.

Then, from the files/ folder, which holds model.py and machine_failure_model.joblib, run python model.py predict your_file.csv.

The CSV needs these 6 columns: Type, Air temperature [K], Process temperature [K], Rotational speed [rpm], Torque [Nm] and Tool wear [min].

Blank cells are filled in automatically. The results are saved to predictions_your_file.csv with a failure probability and a Healthy / Potential failure label.

To retrain, put your_file.csv next to model.py and run python model.py train, or add --quick to skip cross-validation. Full details are in README.md